Chat Application System Design (HLD) – SDE-1 Interview Guide

1. Problem Statement

Design a scalable real-time chat application (WhatsApp/Messenger style) supporting one-to-one messaging, history, presence, and offline delivery.

2. Functional Requirements

• User authentication

• One-to-one messaging

• Real-time delivery

• Chat history

• Online/offline presence

• Read receipts

• Typing indicator

• Optional: group chat, media sharing, search, reactions, E2E encryption

3. Non-Functional Requirements

• Low latency

• High availability

• Horizontal scalability

• Reliability

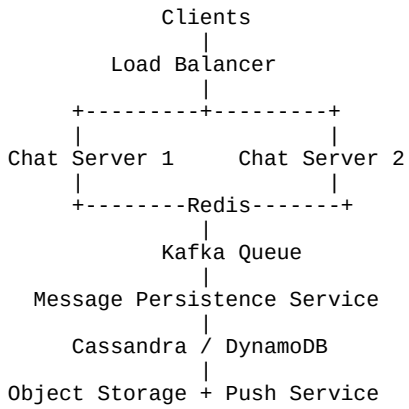
• Durability

• Fault tolerance

4. Back-of-the-Envelope

Example assumptions: 100M DAU, 100 msgs/user/day ⇒ 10B msgs/day (~115K avg QPS, ~250K peak). 150 bytes/message ⇒ ~1.5TB/day (~550TB/year).

5. High-Level Architecture



6. Core Flows

Connection: Client -> LB -> Chat Server -> WebSocket established -> Redis stores UserID -> ServerID

Send Message: Sender -> Chat Server -> Redis lookup -> Receiver's Chat Server -> Receiver

Offline User: Store via Kafka -> DB -> Push Notification -> Fetch missed messages on reconnect

Typing: Sender -> WebSocket -> Receiver (not persisted)

Read Receipt: Receiver opens message -> Read event -> Sender notified

Presence: Heartbeat updates Redis; timeout marks offline

7. Key Components

WebSockets: Persistent low-latency bidirectional communication.

Load Balancer: Distributes new client connections.

Redis: Presence + User→Server mapping (fast in-memory lookups).

Kafka: Durable buffer, retries, decoupling, absorbs traffic spikes.

Cassandra/DynamoDB: Horizontally scalable write-heavy message storage.

Object Storage: Stores media; DB stores URLs.

Push Notifications: Alerts offline users.

8. Scaling

• Stateless chat servers

• Redis Cluster

• Kafka partitions

• DB sharding

• Multi-region deployment

• Object storage for media

9. How to Answer in an Interview

1. Clarify functional and non-functional requirements.

2. Estimate traffic (users, QPS, storage).

3. Start with a simple client-server design.

4. Introduce WebSockets for real-time messaging.

5. Add load balancer and multiple chat servers.

6. Use Redis for routing and presence.

7. Add Kafka for durable asynchronous processing.

8. Persist messages in Cassandra/DynamoDB.

9. Explain offline delivery, read receipts, typing indicators.

10. Finish with scaling, fault tolerance, and trade-offs.

10. Frequently Asked Interview Questions (with concise answers)

Q: Why WebSockets?
A: Persistent full-duplex communication with very low latency.

Q: Why Redis?
A: Fast in-memory presence tracking and user-to-server routing.

Q: Why Cassandra?
A: High write throughput, horizontal scaling, replication.

Q: Why Kafka?
A: Durability, retries, decoupling, spike handling.

Q: How prevent message loss?
A: Persist through Kafka before downstream processing.

Q: How maintain ordering?
A: Partition by conversation and use sequence numbers.

Q: Why not store typing status?
A: Transient event; no business value after delivery.

Q: How deliver offline messages?
A: Persist, notify, sync on reconnect.

Q: How avoid duplicates?
A: Idempotent client message IDs (UUID).

Q: Delete for everyone?
A: Soft delete and broadcast delete event.

Q: E2E encryption?
A: Clients encrypt/decrypt; server stores ciphertext.

Q: Consistency?
A: Strong enough for message durability; eventual for presence/read receipts.

11. Final Cheat Sheet

WebSockets → Real-time messaging
Redis → Presence + routing
Kafka → Durable asynchronous pipeline
Cassandra/DynamoDB → Message storage
Object Storage → Media
Push Service → Offline notifications
Scale → Stateless servers + sharding + partitions + multi-region